

WEEK 9 · LAB

Lab 5: Optimising a Matrix Multiply, in Order

Four changes, each measured on its own, ending with the observation that a library still beats you by a factor of two.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

One 2-hour block

PREREQUISITE

Unit 2, Exercise 02

SUBMIT

lab05-<ROLLNO>.pdf

MARKS

10

OFFERING

Odd Semester 2026-27

The point of this lab

Matrix multiplication is the standard example because every optimisation in the course shows up in it, in a fixed order, and each one is measurable on its own.

The order matters more than the individual tricks. You will confirm that the **serial** improvements come first and are worth more than parallelising the unimproved version.

The rules

- One change at a time. Measure after each.
- State what you expect **before** you measure.
- Same problem size throughout: $n = 2048$, `double`.
- Timing discipline from Lab 1: warm-up, five runs, minimum, spread.
- Verify correctness against a reference after every change. A fast wrong answer is worth nothing.

The versions (7 marks)

#	VERSION	WHAT CHANGES
0	Naive <code>ijk</code>	The baseline
1	Reordered <code>ikj</code>	Inner loop walks memory contiguously
2	Blocked	Working set fits in cache
3	Blocked + OpenMP	Parallelise the outer loops
4	<code>-march=native</code>	Let the compiler use the full instruction set
5	A tuned BLAS	<code>cblas_dgemm</code> from OpenBLAS

For each, report: time, GFLOP/s, speedup over version 0, and percentage of the attainable ceiling from your Lab 3 roofline.

Notes on each step

Version 1. Swap the two inner loops. Explain, in terms of cache lines, why `B[k][j]` with j innermost behaves differently from `B[k][j]` with k innermost.

Version 2. Choose the block size by measurement. Try 16, 32, 64, 128 and report all four. State the working-set calculation that predicts the best one, and whether the prediction held.

Version 3. Parallelise the **outer** loops with `collapse(2)`. Explain why parallelising the innermost loop instead would be a mistake.

Version 4. Check whether the compiler actually vectorised:

```
gcc -O2 -march=native -fopt-info-vec-optimized -c matmul.c
```

Version 5.

```
sudo apt install libopenblas-dev      # or: module load openblas
gcc -O2 matmul_blas.c -o m -lopenblas
```

The analysis (3 marks)

- A1.** Which single change gave the largest gain? Was it a parallel change or a serial one?
- A2.** Compute the product of the serial speedups (versions 1 and 2) and the parallel speedup (version 3). Does the combination behave multiplicatively?
- A3.** Suppose you had parallelised version 0 directly, without versions 1 and 2. Measure that. How does it compare with version 3? State the lesson in one sentence.
- A4.** BLAS should beat your best version by roughly a factor of two. Give two reasons, and answer: in production code, what should you actually write?

PITFALL · THIS EXERCISE IS TO TEACH YOU WHY BLOCKING WORKS

It is not an argument for writing your own GEMM. Real code calls OpenBLAS, MKL, AOCL or cuBLAS. Understanding the technique is the point; reimplementing it in production is not.

What to submit

lab05-<ROLLN0>.pdf : the versions table, the block-size sweep, the vectorisation report excerpt, your four analysis answers, and source as text.

MARKS	FOR
7	All six versions measured correctly, with predictions stated beforehand
3	The analysis, especially A3 and A4